UniPar: A Unified LLM-Based Framework for Parallel and Accelerated Code Translation in HPC

Tomer Bitan[1], Tal Kadosh[2,3], Erel Kaplan[1], Shira Meiri[1], Le Chen[4], Peter Morales[5], Niranjan Hasabnis[5], Gal Oren[1,6]

[1]Technion, [2]Ben-Gurion University, [3]IAEC, [4]Argonne National Laboratory, [5]Code Metal, [6]Stanford University

tomerbitan@cs.technion.ac.il, talkad@post.bgu.ac.il, {erel.kaplan,shira.meiri}@campus.technion.ac.il, lechen@anl.gov, {peter, niranjan}@codemetal.ai, galoren@stanford.edu

2025 IEEE High Performance Extreme Computing Conference (HPEC) | 979-8-3315-7844-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPEC67600.2025.11196677

***Abstract*—Translating programs between various parallel programming languages is an important problem in the high-performance computing (HPC) community, with implications for industry and academia. Existing tools for this problem are either too narrow in scope (translate between specific languages) and/or outdated (requiring maintenance). Recent explosive growth in the popularity of large language models (LLMs) and their ability to generate and translate code offers a potential alternative approach. Toward that end, we first need to systematically evaluate the ability of LLMs to translate between parallel languages.**

In this work, we introduce UniPar, a systematic evaluation framework for LLM-based parallel code translation. Specifically, in this work, we target translations between serial code, CUDA, and OpenMP. Our goal is to assess how well current instruction-tuned LLMs – specifically GPT-4o-mini and LLaMA-3.3-70B-Instruct – can be used out of the box or enhanced through known strategies. We evaluated four major usage modes: hyperparameter optimization for decoding, zero- and few-shot prompting, supervised fine-tuning, and iterative feedback through compiler-based repair. As a part of the evaluation, we construct a new dataset called ParaTrans, covering both serial-to-parallel translation and cross-paradigm transformations.

Our findings reveal that while off-the-shelf models struggle under the default settings (e.g., GPT-4o-mini achieves only 46% compilation and 15% functional correctness), our UniPar methodology – combining fine-tuning, hyperparameter tuning, and compiler-guided repair – improves performance by up to 2X (69% compilation and 33% correctness). We believe that our findings will provide useful insights for researchers to further improve LLMs for the parallel language translation problem.

UniPar source code and ParaTrans dataset are available at our GitHub repository.

I. Introduction

Code parallelization is essential across many high-performance computing (HPC) domains, including climate modeling, molecular dynamics, and astrophysics [1]–[3]. To support it, the HPC community and industry have developed various parallel programming languages and frameworks which have seen significantly expanded adoption in recent years [4] including proprietary options such as CUDA [5] (Nvidia), HIP [6] (AMD), as well as open (not proprietary) languages such as OpenMP [7], SYCL [8] and OpenCL [9]. Proprietary languages, created by hardware vendors to target their devices, create a "lock-in" effect, restricting code to specific hardware. In contrast, open languages offer compatibility across vendors. However, the freedom to easily transfer between these frameworks is highly beneficial in maximizing the best hardware for a task [10]. Whether it be due to features, overall performance, or availability, the ability to easily port code from one parallel computing platform to another grants freedom to researchers and industry alike.

Given the importance of translating between different parallel programming languages, many academic and industry tools have attempted to solve this challenge [11], [12]. Recent examples such as the SYCLomatic (CUDA code into SYCL code) and OpenACC-to-OpenMP5 translator from Intel, along with HIPify (CUDA code into HIP code) from AMD, are such tools [13]–[16]. Academic tools, on the other hand, typically approach the problem as a research problem and offer a wider applicability [17]–[19]. CU2CL, SnuCL and TransCL are such tools for CUDA to OpenCL translation. Our evaluation of these tools found that static tools quickly become outdated as they are not maintained for updates to compiler tool chains and APIs. Other tools are limited in scope, allowing one to translate exclusively from a single API to a single target API.

More recently, industry-academic efforts such as [20]–[28] have explored applying machine learning, including small, specialized language models, to address translation tasks between parallel programming APIs. While these approaches offer greater adaptability than static tools in aspects like supporting new APIs or software versions, they still present a relatively high barrier to entry for practical adoption. Motivated by these limitations, we explore an alternative approach that retains the flexibility of learning-based solutions while improving ease of integration and modification as new models become available.

Large language models (LLMs) have recently demonstrated impressive capabilities in code generation and translation between different programming languages [21], [29], [30], and have been widely studied for their applicability in HPC domains [31]–[36]. The natural question then is: *How effective are popular LLMs in translating code between different parallel programming languages?* Studies have demonstrated that LLMs exhibit an understanding of parallel programming APIs [37]–[39], including work by Nichols et al. [40]–[43] that evaluated LLMs for serial and parallel code generation tasks. Nevertheless, to the best of our knowledge, no existing

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.

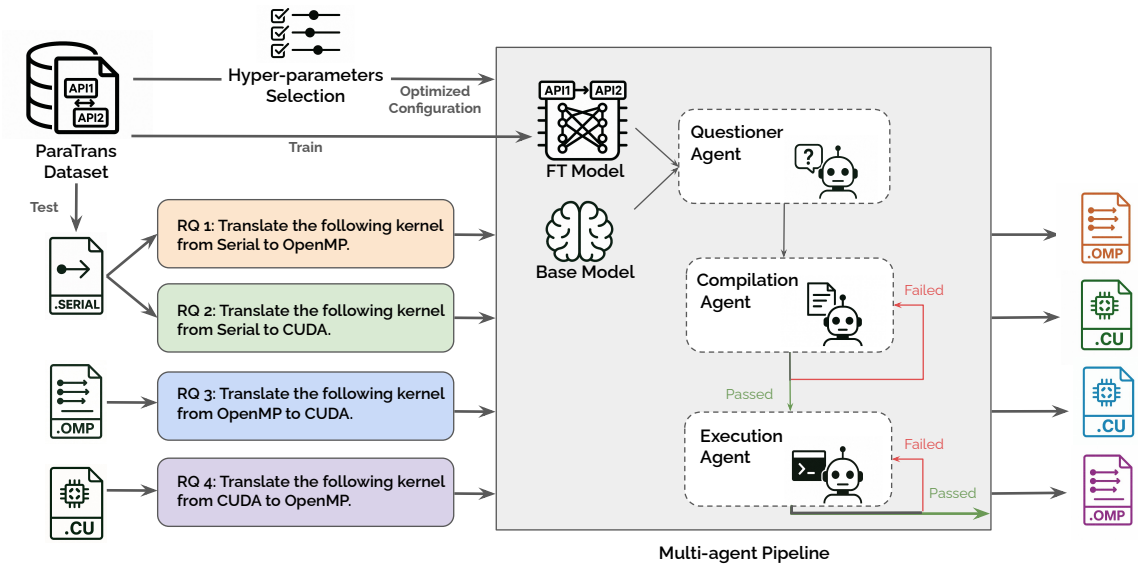


Fig. 1: UniPar Architecture — Multi-Agent LLM Pipeline for Parallel Code Translation. The system translates kernel functions from the ParaTrans dataset (covering Serial, OpenMP, and CUDA implementations) in one of four directions: Serial→OpenMP, Serial→CUDA, OpenMP→CUDA, or CUDA→OpenMP. A base or fine-tuned LLM generates candidate translations, optionally using optimized decoding hyperparameters. The multi-agent pipeline then refines these outputs through three components: (1) the *Questioner Agent*, which issues the translation prompt; (2) the *Compilation Agent*, which attempts to compile the generated code and repairs it using compiler diagnostics; and (3) the *Execution Agent*, which validates runtime correctness and triggers additional repair attempts if necessary. This feedback-driven loop continues for up to three rounds. The final output is a syntactically and semantically correct kernel in the desired target API.

work evaluates the ability of LLMs to translate code between multiple parallel programming languages with comprehensive techniques to improve their performance.

In this paper, we provide the first systematic evaluation of LLMs (specifically LLaMA-3.3-70B-Instruct and GPT-4o-mini) in their ability to translate between multiple parallel programming languages (namely CUDA and OpenMP). Towards that end, we first develop a cross-paradigm code corpus named ParaTrans and apply it to the baseline evaluation of the LLMs. Additionally, we evaluate four commonly employed improvement strategies built around the LLMs: hyperparameter optimization for decoding, zero- and few-shot prompting, supervised fine-tuning, and iterative feedback through compiler-based repair. Specifically, for every strategy, we evaluate its effects on the model performance and determine if its effects are independent and additive.

Research Questions: We pose the following RQs:

- **RQ1 (Baseline Inference):** Can the base ("vanilla") instruction-tuned models effectively perform parallel code translation using zero-shot and few-shot prompting (with the best hyperparameters)?
- **RQ2 (Supervised Fine-tuning):** Does supervised fine-tuning on the code translation task improve the model's ability to generate correct and compilable parallel code?
- **RQ3 (Agentic Pipeline):** What is the impact of integrating LLMs in an agent-based framework (that provides the compiler feedback) on the translation quality?

Contributions: This paper makes the following contributions:

1) To the best of our knowledge, UniPar is the first attempt at a systematic evaluation of LLMs on the problem of code translation between parallel programming languages.
2) We construct the first standardized dataset for translation between parallel programming paradigms (Serial, OpenMP, and CUDA), enabling a systematic evaluation.
3) We evaluate three different commonly employed strategies to improve the performance of LLMs, namely, few-shot prompting, fine-tuning, and agentic pipelines. The results reveal that although few-shot prompting is ineffective for this problem, both task-specific fine-tuning and agentic framework with compiler feedback deliver considerable improvement to the performance of LLMs, by up to 2X (69% compilation and 33% correctness).

II. Our Approach: UniPar

We explore three complementary strategies (shown in Figure 1) to evaluate the ability of an LLM to translate between parallel programming paradigms. First, we assess the foundation models in the zero- and few-shot settings, exploring hyperparameters (e.g., temperature, max token) for optimal performance. Second, we fine-tune an LLM on our curated dataset of aligned kernel implementations, tailoring the model to the translation task. Finally, we develop an agentic pipeline that integrates the LLM with a compiler and an output verifier for an iterative refinement.

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.

We used our SOTA cross-paradigm code translation corpus, ParaTrans, to benchmark our methodology (section III).

III. ParaTrans: Cross-Paradigm Code Translation Corpus

To construct a parallel dataset designed to translate between heterogeneous parallel programming languages (e.g., OpenMP and CUDA) and to parallelize serial code, we leveraged the HeCBench collection of computing benchmarks [44]. HeCBench suite comprises 499 benchmarks that cover a wide range of computational domains such as cryptography, machine learning, signal processing, and more. Each benchmark is implemented in multiple parallel programming paradigms, including CUDA, HIP, SYCL/DPC++, and OpenMP-4.5, resulting in 1758 benchmark implementations (as not all benchmarks are available in every parallel programming language).

Source Scraping. We applied several filtering criteria to curate the ParaTrans corpus. First, since our focus is exclusively on OpenMP and CUDA, only benchmarks implemented in at least one of these two paradigms were retained. Second, to enable accurate and meaningful translation, it was necessary to isolate the core computational logic of each benchmark, typically encapsulated in a single source file. As our objective is to translate only the core functionality between parallel languages, we analyzed the source code structure of each benchmark. 706 out of 1758 programs consist of a single source file, which clearly contains the primary computational logic. In benchmarks with multiple source files, we observed that auxiliary files such as those named 'utils' or 'reference' did not contain the main functionality and were therefore excluded. Applying these filters resulted in 406 CUDA-based codes and 276 OpenMP-based codes.

Preprocessing. To standardize the code samples between paradigms, we applied a preprocessing step. First, all comments were removed to reduce noise and focus solely on the code logic. Then, we generated serial versions of the kernels by utilizing the OpenMP-based implementations and removing OpenMP pragmas from them.

Code Verification. To ensure the quality of the dataset, we applied an automatic validation process to the benchmark kernel codes. This process verifies that each code sample compiles successfully as well as produces outputs consistent with the expected ground-truth results. Kernel implementations that failed this validation step were excluded from the dataset. Note, three of the benchmark kernels did not pass validation and were therefore removed from subsequent experiments.

Token Count Pruning. Prompting LLMs with entire source codes can result in significant token imbalance across prompts. Prior studies have shown that LLM performance may degrade when processing long or unbalanced inputs, as the context window becomes saturated or loses focus [45]. Moreover, shorter programs often yield more accurate and coherent model output. To address this, we applied a token-length pruning strategy to standardize input lengths and filter out excessively long samples. We set a cut-off of 7,500 tokens, determined empirically by analyzing the token distribution of source files using the LLaMA-3.3-70B-Instruct tokenizer. This threshold effectively balances input coverage and model context fidelity. After applying this constraint, we retained 380 CUDA-based and 258 OpenMP-based programs in ParaTrans.

TABLE I: The training and test splits of ParaTrans.

Translation	# Train
Serial → OpenMP	235
Serial → CUDA	221
CUDA → OpenMP	221
OpenMP → CUDA	221
Total	**898**

Translation	# Test
Serial → OpenMP	20
Serial → CUDA	19
CUDA → OpenMP	18
OpenMP → CUDA	19
Total	**76**

Altogether, ParaTrans is composed of aligned tuples, with each tuple containing implementations of the same kernel in different parallel programming languages. These tuples enable translation between any pair of languages among Serial, OpenMP, and CUDA. To support a fair evaluation, we split the dataset into a training set and a held-out test set, as shown in Table I. The training set contains 898 translation pairs, while the test set comprises 76 pairs, resulting in an approximate ratio of 9 to 1.

IV. Evaluation

A. Evaluation Metrics

To evaluate the capabilities of LLMs in generating parallel code, we adopt a validation methodology inspired by prior work [46]–[48], leveraging HeCBench's support for the automated compilation and execution with output verification. Specifically, we evaluate model outputs on the held-out test split of the ParaTrans dataset. This framework enables assessment of both the *compilation rate* – the percentage of generated programs that compile successfully – and the *validation rate*, the percentage of generated programs producing outputs that match the expected output specified in the HeCBench kernel. For validation, we replaced the `main` function in the generated code with the `main` function in the ground-truth kernel, as the `main` function in the ground-truth kernel contained the output verification check. In places where the code compiled successfully before the replacement but failed after the replacement, we used 3 attempts with GPT-4o-mini to repair the compilation error. We then verified that no changes were made that affect kernel functionality.

B. Experimental Setup

We evaluated the performance of two instruction-tuned foundation models: GPT-4o-mini [49] (referred to as GPT in short in the paper) and LLaMA-3.3-70B-Instruct [50] (referred to as LLaMA in short in the paper). These models are selected for their accessibility and strong performance on a wide range of general-purpose code generation tasks, without requiring additional fine-tuning [51], [52].

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.

LLM Hyperparameters. LLMs include several hyperparameters that significantly affect their outputs. The main ones are *temperature*, *top-p*, and *max tokens*. *Temperature* adjusts the randomness (aka "creativity"): higher values lead to more diverse outputs, while lower values make responses more deterministic. *Top-p* (nucleus sampling) sets a cumulative probability threshold, retaining only the most likely tokens whose total probability exceeds the cutoff. *Max tokens* limits the number of tokens the model may generate in its response (excluding the prompt). After empirical tuning (see subsection V-A), we found that a *temperature* of 0.2 and a *top-p* of 0.9 provided reliable results across both models. We used a maximum token limit of 15,000 whenever possible to accommodate the widest range of model outputs.

LLM Quantization. To fine-tune the model, we used 16-bit floating point precision, which offers a good balance between computational efficiency and training stability [53]. For inference, we applied 8-bit quantization to reduce memory usage and improve latency without significantly impacting model performance [54].

Hardware. For LLaMA model inference, we used four NVIDIA L40 GPUs and twelve Intel Xeon Gold 6336Y CPUs. GPT-4o-mini inference was performed via the Azure OpenAI API [55]. For the verification evaluation, OpenMP code was executed on a machine equipped with fifty AMD EPYC 7742 CPUs, and CUDA code was run on the same NVIDIA L40 GPUs mentioned above. Fine-tuning was conducted using eight NVIDIA A100 GPUs (80GB).

V. Results and Analysis

In this section, we first answer RQ1, exploring different temperatures to present the baseline performance of the chosen LLMs in the zero-shot (subsection V-A). Next, we examine the few-shot prompt settings (subsection V-B). We then evaluate the effectiveness of supervised fine-tuning (subsection V-C) and the agentic compilation repair pipeline (subsection V-D) on the baseline performance of the models.

A. *RQ1*: Translation Behavior with Zero-shot Prompting

We evaluated both LLaMA-3.3-70B-Instruct and GPT-4o-mini across a range of decoding configurations. Specifically, we tested three *temperature* values (0.2, 0.6, and 0.9) and three *max token* limits (5,000, 10,000, and 15,000), while keeping the *top-p* value fixed at 0.8 in a zero-shot prompting setup. Empirically, varying *top-p* did not produce significant improvements in performance, and with a very slight improvement when switching from 0.8 to 0.9. was therefore held constant.

As shown in Figure 2, lower *temperature* settings yielded both higher and more consistent results across models, with the largest effect observed in the configurations allowing longer outputs. This suggests that reduced randomness in the token selection process improves syntactic validity, especially when generating larger kernel codes, as supported by the findings in previous work [56], [57]. This finding is particularly important when accommodating a broader variety of kernels, such as more complex or larger kernels. The rest of our experiments were all done with a temperature of 0.2, top-p of 0.9, and max output tokens set to 15000.

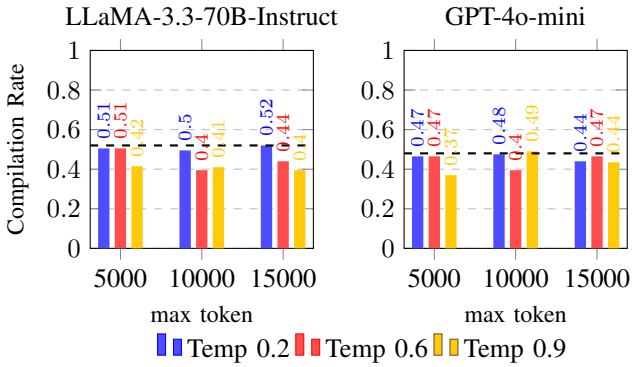


Fig. 2: Effect of Temperature and Max Token Settings on Compilation Rate. Compilation success rates for the models across combinations of temperature and maximum token limits. Lower temperatures, particularly 0.2, yield the highest and most consistent compilation rates, with the effect especially pronounced at higher token lengths for LLaMA.

B. *RQ1*: Translation Behavior with Few-shot Prompting

LLMs are trained on a broad range of general tasks. In many scenarios, providing input-output examples during inference (referred to as few-shot prompting) can help guide the model towards the specific task at hand [58], [59]. To assess its effect in our setting, we compared the compilation rates of generated code with 0-shot, 1-shot, 2-shot, and 3-shot configurations.

Each *n*-shot involved appending *n* additional examples for the same source–target language pair from the ParaTrans train set. We used the prompt addition shown in Figure 3. Each example consists of a source kernel and its corresponding translation to the target language, with both the original and the target language specified.

Instruction Prompt for Inference

System: You are an HPC expert specializing in translating between parallel programming APIs.

Addition for 1-Shot prompt:
Instruction: Translate the following code from {from_api} to {to_api}
Code: {from_code}
Assistant: Here is the translated code: {to_code}

Instruction: Translate the following code from {from_api} to {to_api}
Code: {from_code}
Response: {to_code}

Fig. 3: Instruction Format for Translation Inference. The prompt template used to guide the model during translation between parallel programming schemes, including optional one-shot augmentation with an example input-output pair.

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.

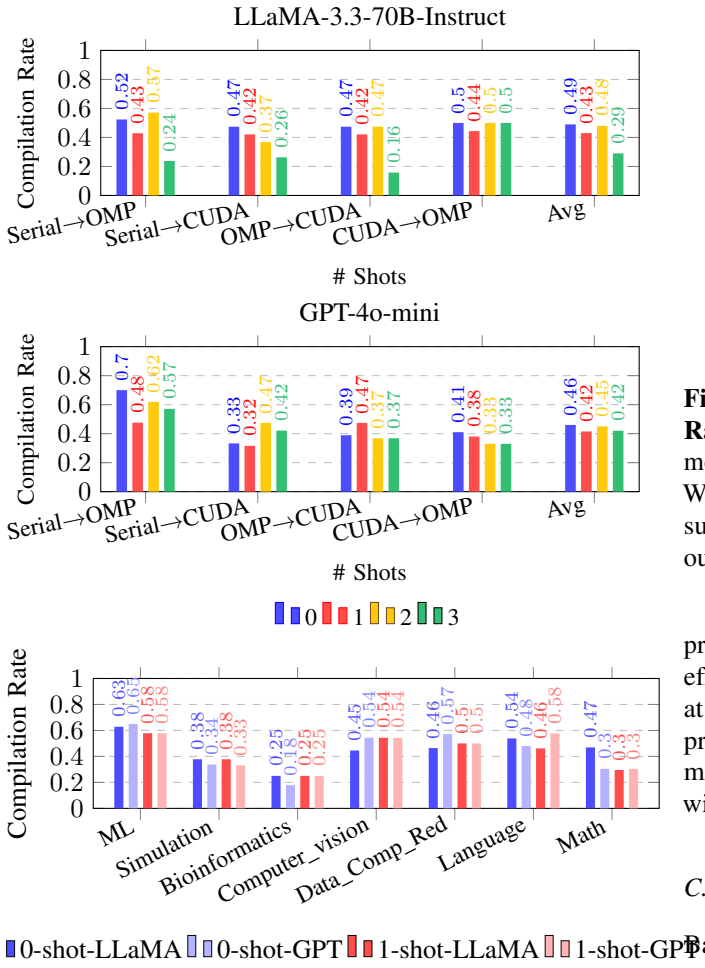


Fig. 4: Effect of Few-Shot Prompting on Compilation Rate. Compilation rates for the models across 0- to 3-shot configurations and for different HECBENCH categories. Increasing the number of prompt examples did not consistently improve the compilation rate and can lead to degradation, especially beyond 1-shot. Effects vary by translation direction and application domain.

The results of this experiment are shown in Figure 4. X axis shows the number of shots provided to the model, while the Y axis shows the compilation rate. The results reveal that both models exhibit minimal sensitivity to the addition of few-shot examples. Compilation success rates remain largely unchanged, except for 3-shot prompting, where performance degrades noticeably. We investigated this behavior further, splitting the task into the topic categories seen in HECBENCH (e.g., machine learning, math, etc.). We saw that more shots typically did not help the examples in all categories.

While the exact cause of the poor LLM performance with the few-shot prompting would warrant further investigation, specific examples reveal that even on samples similar or identical to the one-shot prompt, LLaMA-3.3-70B-Instruct still introduces subtle but critical syntactic errors. We suspect this results from a combination of context dilution with longer

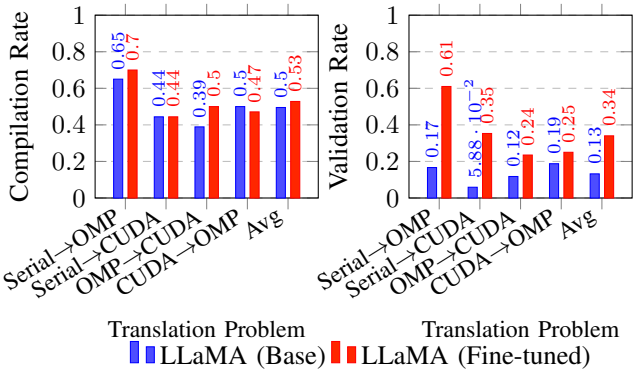


Fig. 5: Effect of Fine-Tuning on Compilation and Validation Rates. Compilation and validation rates for the LLaMA model across translation directions before and after fine-tuning. While compilation rate improvements are modest, fine-tuning substantially increases the proportion of functionally-correct outputs, with average validation rates more than doubling.

prompts and the nature of few-shot prompting, which is most effective at guiding answers' high-level structure and less at capturing fine-grained details within that structure. This prompted us to seek a method to more robustly shape the model's domain knowledge, which led us to the experiment with the fine-tuning.

C. *RQ2: Supervised Fine-Tuning*

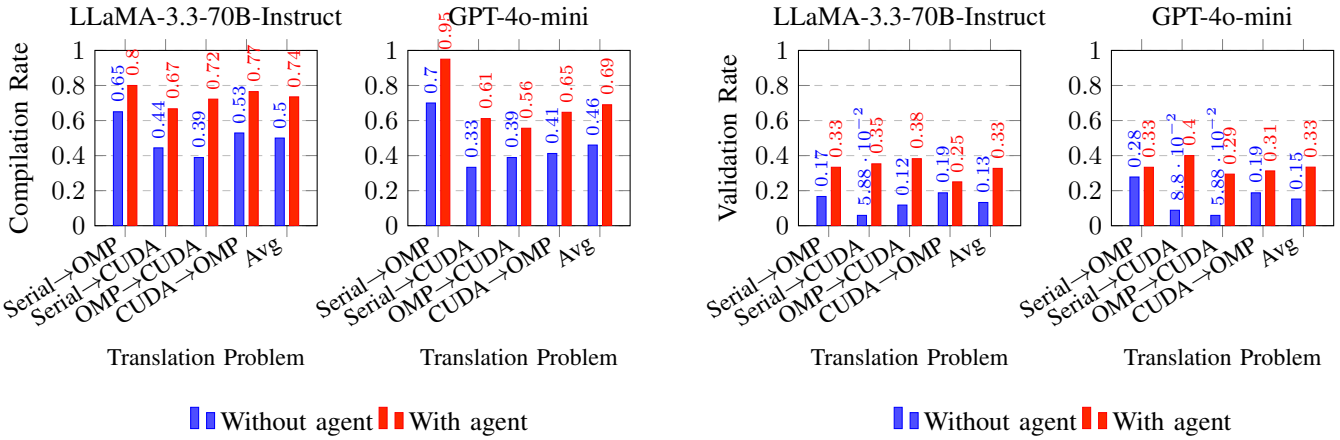
Base instruction-tuned models such as GPT and LLaMA are trained for a general use. To adapt these models specifically for the task of parallel code translation, we fine-tuned them on the PARATRANS training set to narrow their scope and enhance their performance [25], [46], [60], [61].

We used the instruction-tuned version of LLaMA-3.3-70B-Instruct model, leveraging the alignment already present to accelerate convergence and improve performance, as instruction-tuned models require less supervision to adapt to new tasks compared to fine-tuning from a base model [62], [63]. We trained the model on aligned kernel tuples from PARATRANS. The prompt format used in training is shown in Figure 3.

To accommodate source codes spanning thousands of tokens, we maximized the model's context length within the hardware memory constraints. Using memory optimization techniques provided by the TorchTune library [64], we configured the model to operate with the context length of 16,384 tokens.

Figure 5 shows the performance of the base and the fine-tuned LLaMA model across all translation problems on the held-out test set of the PARATRANS corpus. The results reveal that the fine-tuned model shows a performance that is on par with or exceeds that of the base models in terms of the compilation rate. In particular, the fine-tuned model outperforms LLaMA-3.3-70B-Instruct in generating semantically valid code, achieving a validation rate of 29.4% for translating to CUDA and 38.2% for translating to OpenMP.

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.



(a) Impact of Agentic Pipeline on Compilation Rate Across Translation Problems. Compilation rates for Questioner LLMs across all translation problems, with and without the agentic pipeline (Compilation and Execution agents). Agent-based repair has a slight equalizing effect on results across translation directions as the variation in compilation rate shrinks.

(b) Impact of Agentic Pipeline on Validation Rate Across Translation Problems. Validation rates for Questioner LLMs across all translation problems, with and without the agentic pipeline (Compilation and Execution agents). Agent-based repair substantially improves functional correctness in every direction, including challenging cases such as OMP→CUDA and Serial→CUDA.

Fig. 6: Impact of Agentic Pipeline

D. *RQ3: Agentic Translation with Compiler Feedback*

To further enhance LLM-based translation, we implemented a multi-agent, feedback-driven architecture, aimed at transforming the translation process from a static process to a dynamic pipeline that mirrors the work of human programmers. The system refines LLM-generated translations through structured iterations grounded in the feedbacks from a compiler and an output verifier. This pipeline, shown in Figure 1, is inspired by prior work [65]–[69], but differs in that we evaluate its utility purely during inference, rather than for the dataset generation.

In our multi-agent pipeline, each agent has access to an LLM model and external tools, including output verifiers and compilers, such as *Clang++* and *NVCC*. Agents rely solely on the current translation status and the available context of the environment for decision-making. The argentic pipeline consists of three agents orchestrated in a feedback cycle:

- **Questioner Agent**: Assembles the initial prompts that define the source and target languages (e.g., CUDA → OpenMP). This role can be fulfilled by a vanilla model or its improved version, such as a fine-tuned model.
- **Compilation Agent**: Attempts to compile the generated code. If the compilation fails, it extracts the error message and formulates a compilation repair prompt for its model.
- **Execution Agent**: Executes the code that passes compilation. If output verification fails, it produces an output-repair prompt containing the observed behavior and invokes its model for a correction.

Our experiment focused on the use of feedback from tools to improve models' performance. Thus, the pipeline results we report are for a very straightforward pipeline in which the questioner agent first performs an initial translation. Then the compilation agent has three attempts to compile and correct the code according to any errors revealed by the compiler. Finally, if the compilation succeeds, the execution agent has three attempts to run the code on the test cases supplied in the original HECBENCH code and correct any errors that arise from the feedback from the runtime environment. Note that in this stage, instead of the pass/fail status of the test, we feed the runtime errors for program repair.

In our experiments, we evaluated both LLaMA-3.3-70B-Instruct and GPT-4o-mini as the Questioner agent. Both the Compilation and Execution agents used 3 rounds with GPT-4o-mini as their underlying LLM. All LLMs were configured with a *temperature* of 0.2, *top-p* of 0.9, and a *max token* limit of 15,000. While a number of setup configurations are possible for this experiment, we have reserved that study for future work.

Figure 6 shows the overall impact of the agentic pipeline on various translation problems, with Figure 6a and Figure 6b showing the performance of the pipeline in terms of compilation and validation rates resp. Specifically, the legend `Without Agent` refers to the performance without agentic pipeline, while `With Agent` refers to the performance with the agentic pipeline. In general, incorporating compiler and output verifier feedback significantly improves overall performance in terms of compilation and validation rate. Repeated rounds of prompting, each incorporating additional diagnostic information, enabled the system to repair a variety of code failures.

Ablation Study: Effectiveness of different Questioner Models. We evaluated different Questioner models to understand their impact on the performance (shown in Figure 7). We make observation similar to above in this case also — agentic pipeline has considerable impact on all the Questioner Models. Furthermore, the improvement appears additive, as even when starting from a fine-tuned base model, which already outperforms the vanilla

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.

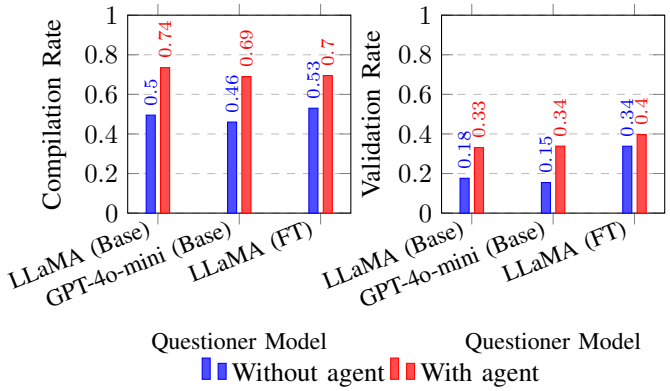


Fig. 7: Overall Impact of the Agentic Pipeline Across Different Questioner Models. Compilation and validation rates for various Questioner models with and without the agentic pipeline (Compilation and Execution agents). Incorporating Compilation and Execution agents leads to a substantial improvement in both metrics, especially for base models. While the fine-tuned model shows smaller gains, it consistently achieves the highest overall validation rate.

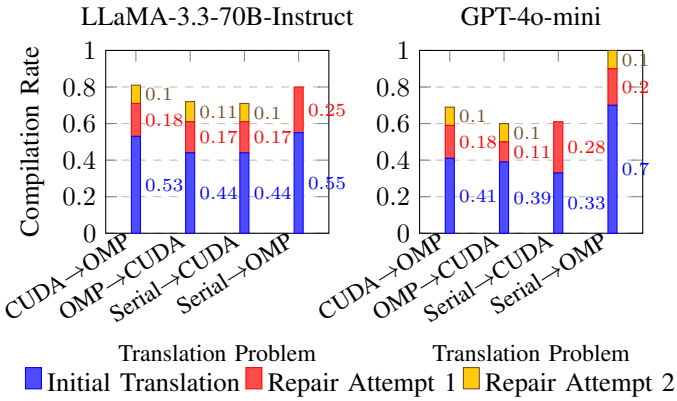


Fig. 8: Effect of Compilation Agent's Repair Attempts on Compilation Rate. Compilation rates for the models across translation directions, broken down by initial generation and up to two rounds of agent-guided repair. The first repair attempt yields the largest improvement, particularly in directions with initially low success rates. Additional gains from the second repair attempt help equalize performance across translation tuples.

zero-shot baseline, the feedback loop improves performance even further. Although the improvement in the compilation rate is not as pronounced as the improvement in the validation rate, we observe that the fine-tuned model maintains its advantage.

Ablation Study: Effectiveness of Compilation Agent in repairing errors. To analyze the effectiveness of the agentic repair pipeline, we plotted the compilation rate improvements for different repair rounds. Figure 8 shows the performance breakdown across different repair rounds and translation problems. We observed that in most of these cases, the first repair attempt provides the largest compilation rate improvement. This stems from the behavior of the model used by the compilation agent that manages to solve only one of two types of issues in each iteration, so codes with multiple needed corrections require more than one iteration. While our current experiments show limited gains from the second iteration, this pattern suggests that longer or more complex code samples may benefit more substantially from additional repair rounds. Furthermore, the gains are not distributed entirely uniformly across translation tasks. Specifically, the pipeline provides slightly larger improvements on the translation problems where the base model initially struggled, such as OMP→CUDA in LLaMA or Serial→CUDA on GPT. This equalizing effect suggests that the agent-based system helps surface translation capabilities that are hidden in static prompting setups. The high performance in Serial→OMP stems from the close similarity between the two, both inherently and due to the generation method described in section III.

This result highlights that significant improvements in translation quality can be unlocked via inference time improvements alone, especially when combined with complementary improvements such as fine-tuning.

VI. Future Work

Though this preliminary paper performs systematic evaluation of LLMs of their ability to translate between parallel programming languages, we acknowledge that we have not covered a few important problems in this paper. We mention a few of them here. Although we checked the syntax and the functional correctness of the translated code, we did not check its runtime performance or scalability – factors that are typically important for an HPC application – nor did we ask LLMs to optimize them. Furthermore, we have reserved an in-depth ablation study along with an evaluation of various design choices for the future. Moreover, current study is restricted to translations to CUDA and OpenMP and its applicability to other important HPC languages such as SYCL, OpenACC, etc., is an interesting question. We will continue to explore some of these problems and encourage the HPC community to join us as well.

VII. Conclusion

In this paper, we introduced UniPar, a unified methodology for generating parallel code as well as translating between a variety of parallel-programming APIs with one system. The framework unifies several enhancements atop a base LLM: (1) exploration of key hyperparameters, including temperature selection and the less effective few-shot inference; (2) fine-tuning on parallel code data; and (3) integration into a dynamic, feedback-driven pipeline. Each component contributes independently to compilation and validation success, with their effects compounding when combined. While UniPar demonstrates significant improvements, further work is needed to enhance compilation and validation rates and to broaden support for longer kernels and a wider range of APIs.

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.

ACKNOWLEDGMENTS

This research was supported by the Pazy Foundation and the Lynn and William Frankel Center for Computer Science. Computational support was provided by Foundry Cloud Platform and Code Metal.

REFERENCES

[1] J. W. Hurrell, M. M. Holland, P. R. Gent, S. Ghan, J. E. Kay, P. J. Kushner, J.-F. Lamarque, W. G. Large, D. Lawrence, K. Lindsay *et al.*, "The community earth system model: a framework for collaborative research," *Bulletin of the American Meteorological Society*, vol. 94, no. 9, pp. 1339–1360, 2013.

[2] J. C. Phillips, R. Braun, W. Wang, J. Gumbart, E. Tajkhorshid, E. Villa, C. Chipot, R. D. Skeel, L. Kale, and K. Schulten, "Scalable molecular dynamics with namd," *Journal of computational chemistry*, vol. 26, no. 16, pp. 1781–1802, 2005.

[3] G. L. Bryan, M. L. Norman, B. W. O'Shea, T. Abel, J. H. Wise, M. J. Turk, D. R. Reynolds, D. C. Collins, P. Wang, S. W. Skillman *et al.*, "Enzo: An adaptive mesh refinement code for astrophysics," *The Astrophysical Journal Supplement Series*, vol. 211, no. 2, p. 19, 2014.

[4] T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Quantifying openmp: Statistical insights into usage and adoption," 2023.

[5] "CUDA C++ Programming Guide," https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html, 2023, [Online].

[6] ROCm Developer Tools Team, "HIP: Heterogeneous-compute interface for portability," https://github.com/ROCm-Developer-Tools/HIP, 2025, accessed: 2025-06-09.

[7] T. Mattson, Y. H. He, and A. Koniges, *The OpenMP Common Core: Making OpenMP Simple Again (Scientific and Engineering Computation)*. The MIT Press, 2019.

[8] R. Reyes and V. Lomüller, "SYCL: Single-source C++ accelerator programming," in *Parallel Computing: On the Road to Exascale*. IOS Press, 2016.

[9] A. Munshi, "The opencl specification," in *2009 IEEE Hot Chips 21 Symposium (HCS)*. IEEE, 2009, pp. 1–314.

[10] S. J. Pennycook, S. D. Hammond, S. A. Wright, J. Herdman, I. Miller, and S. A. Jarvis, "An investigation of the performance portability of opencl," *Journal of Parallel and Distributed Computing*, vol. 73, no. 11, pp. 1439–1450, 2013.

[11] R. Harel, I. Mosseri, H. Levin, L.-o. Alon, M. Rusanovsky, and G. Oren, "Source-to-source parallelization compilers for scientific shared-memory multi-core and accelerated multiprocessing: analysis, pitfalls, enhancement and potential," *International Journal of Parallel Programming*, vol. 48, pp. 1–31, 2020.

[12] I. Mosseri, L.-o. Alon, R. Harel, and G. Oren, "Compar: optimized multi-compiler for automatic openmp s2s parallelization," in *OpenMP: Portable Multi-Level Parallelism on Modern Systems: 16th International Workshop on OpenMP, IWOMP 2020, Austin, TX, USA, September 22–24, 2020, Proceedings 16*. Springer, 2020, pp. 247–262.

[13] R. Mueller-Albrecht, "Syclomatic: Sycl adoption for everyone-moving from cuda to sycl gets progressively easier: Advanced migration considerations," in *Proceedings of the 12th International Workshop on OpenCL and SYCL*, 2024, pp. 1–2.

[14] H. Servat, G. Rossi, A. Duran, and R. Narayanaswamy, "On the migration of openacc-based applications into openmp 5+," in *International Workshop on OpenMP*. Springer, 2022, pp. 127–141.

[15] Y. Fridman, Y. Goren, and G. Oren, "From openacc to openmp5 gpu offloading: Performance evaluation on nas parallel benchmarks," in *Proceedings of the 2025 4th International Workshop on Extreme Heterogeneity Solutions*, 2025, pp. 10–18.

[16] ROCm Developers, "HIPIFY: Cuda to hip source translation tool," https://github.com/ROCm/HIPIFY, 2023, accessed: 2025-06-09.

[17] A. Head, "cu2cl: Cuda to opencl source-to-source translator," https://github.com/andrewhead/cu2cl, 2013, accessed: 2025-06-09.

[18] J. Kim, S. Seo, J. Lee, J. Nah, G. Jo, and J. Lee, "Snucl: an opencl framework for heterogeneous cpu/gpu clusters," in *Proceedings of the 26th ACM international conference on Supercomputing*, 2012, pp. 341–352.

[19] C. Shi, Y. Sun, R. Chen, J. Wang, Q. Guo, C. Gong, Y. Sui, Y. Jin, and Y. Zhang, "Transcl: An automatic cuda-to-opencl programs transformation framework," *ACM Transactions on Architecture and Code Optimization*, 2025.

[20] Q. I. Mahmud, A. TehraniJamsaz, N. K. Ahmed, T. L. Willke, and A. Jannesari, "Contraph: Contrastive learning for parallelization and performance optimization," *Proceedings of the 2025 ACM International Conference on Supercomputing (ICS '25)*, 2025.

[21] A. Tehrani, A. Bhattacharjee, L. Chen, N. K. Ahmed, A. Yazdanbakhsh, and A. Jannesari, "Coderosetta: Pushing the boundaries of unsupervised code translation for parallel programming," *Advances in Neural Information Processing Systems*, vol. 37, pp. 100 965–100 999, 2024.

[22] T. Kadosh, N. Hasabnis, V. A. Vo, N. Schneider, N. Krien, A. Wasay, N. Ahmed, T. Willke, G. Tamir, Y. Pinter *et al.*, "Scope is all you need: Transforming LLMs for HPC Code," *arXiv preprint arXiv:2308.09440*, 2023.

[23] T. Kadosh, N. Schneider, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Advising openmp parallelization via a graph-based approach with transformers," *arXiv preprint arXiv:2305.11999*, 2023.

[24] N. Schneider, T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "MPI-RICAL: Data-Driven MPI Distributed Parallelism Assistance with Transformers," in *Proceedings of the SC '23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, ser. SC-W '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 2–10. [Online]. Available: https://doi.org/10.1145/3624062.3624063

[25] N. Schneider, N. Hasabnis, V. A. Vo, T. Kadosh, N. Krien, M. Capota, G. Tamir, T. L. Willke, N. Ahmed, Y. Pinter *et al.*, "Mpirigen: Mpi code generation through domain-specific language models," in *Proceedings of the 2024 Workshop on AI For Systems*, 2024, pp. 1–6.

[26] R. Harel, Y. Pinter, and G. Oren, "Learning to parallelize in a shared-memory environment with transformers," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2023, pp. 450–452.

[27] T. Kadosh, N. Hasabnis, V. A. Vo, N. Schneider, N. Krien, M. Capotă, A. Wasay, G. Tamir, T. Willke, N. Ahmed *et al.*, "Monocoder: Domain-specific code language model for hpc codes and tasks," in *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2024, pp. 1–7.

[28] L. Chen, A. Bhattacharjee, N. Ahmed, N. Hasabnis, G. Oren, V. Vo, and A. Jannesari, "OMPGPT: A generative pre-trained transformer model for openmp," *arXiv preprint arXiv:2401.16445*, 2024.

[29] B. Roziere, M.-A. Lachaux, L. Chanussot, and G. Lample, "Unsupervised translation of programming languages," *Advances in neural information processing systems*, vol. 33, pp. 20 601–20 611, 2020.

[30] A. Dhruv and A. Dubey, "Leveraging large language models for code translation and software development in scientific computing," in *Proceedings of the Platform for Advanced Scientific Computing Conference*, 2025, pp. 1–9.

[31] L. Chen, N. K. Ahmed, A. Dutta, A. Bhattacharjee, S. Yu, Q. I. Mahmud, W. Abebe, H. Phan, A. Sarkar, B. Butler *et al.*, "The landscape and challenges of hpc research and llms," *arXiv preprint arXiv:2402.02018*, 2024.

[32] X. Huang, X. Zhang, L. Tao, R. Mao, N. Zhou, W. Zhu, M. Deng, J. Meng, Y. Wei, A. C. Zhou *et al.*, "Paracoder: Parallel code generation with large language model," in *Proceedings of the 1st FastCode Programming Challenge*, 2025, pp. 1–7.

[33] D. Yadav and S. Mondal, "Evaluating pre-trained large language models on zero shot prompts for parallelization of source code," *Available at SSRN 5114036*, 2025.

[34] R. Jain and R. Purandare, "Assessing large language models in comprehending and verifying concurrent programs across memory models," *arXiv preprint arXiv:2501.14326*, 2025.

[35] J. Yin, J. Hines, E. Herron, T. Ghosal, H. Liu, S. Prentice, V. Lama, and F. Wang, "chathpc: Empowering hpc users with large language models," *The Journal of Supercomputing*, vol. 81, no. 1, p. 194, 2025.

[36] Q. I. Mahmud, A. TehraniJamsaz, H. D. Phan, L. Chen, M. Capotă, T. L. Willke, N. K. Ahmed, and A. Jannesari, "Autoparllm: Gnn-guided context generation for zero-shot code parallelization using llms," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 11 821–11 841.

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.

[37] A. Rahman, V. Cvetkovic, K. Reece, A. Walters, Y. Hassan, A. Tummeti, B. Torres, D. Cooney, M. Ellis, and D. Nikolopoulos, "Performance evaluation of large language models for high-performance code generation: A multi-agent approach (marco)," *arXiv preprint arXiv:2505.03906v1*, 2025.

[38] G. Bolet, G. Georgakoudis, H. Menon, K. Parasyris, N. Hasabnis, H. Estes, K. W. Cameron, and G. Oren, "Can large language models predict parallel code performance?" *arXiv preprint arXiv:2505.03988*, 2025.

[39] R. Harel, T. Kadosh, N. Hasabnis, T. Mattson, Y. Pinter, and G. Oren, "Pragformer: data-driven parallel source code classification with transformers," *International Journal of Parallel Programming*, vol. 53, no. 1, pp. 1–26, 2025.

[40] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can large language models write parallel code?" in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*, 2024, pp. 281–294.

[41] D. Nichols, A. Marathe, H. Menon, T. Gamblin, and A. Bhatele, "Hpc-coder: Modeling parallel programs using large language models," in *ISC High Performance 2024 Research Paper Proceedings (39th International Conference)*. Prometeus GmbH, 2024, pp. 1–12.

[42] A. Chaturvedi, D. Nichols, S. Singh, and A. Bhatele, "Hpc-coder-v2: Studying code llms across low-resource parallel languages," in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*. Prometeus GmbH, 2025, pp. 1–14.

[43] D. Nichols, "On learning behaviors of parallel code and systems across modalities," Ph.D. dissertation, University of Maryland, College Park, 2025.

[44] Z. Jin and J. S. Vetter, "A benchmark suite for improving performance portability of the sycl programming model," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 325–327.

[45] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *arXiv preprint arXiv:2307.03172*, 2023.

[46] T. Kadosh, N. Hasabnis, P. Soundararajan, V. A. Vo, M. Capota, N. Ahmed, Y. Pinter, and G. Oren, "Ompar: Automatic parallelization with ai-driven source-to-source compilation," *arXiv preprint arXiv:2409.14771*, 2024.

[47] W. Yan, H. Liu, Y. Wang, Y. Li, Q. Chen, W. Wang, T. Lin, W. Zhao, L. Zhu, H. Sundaram *et al.*, "Codescope: An execution-based multilingual multitask multidimensional benchmark for evaluating llms on code understanding and generation," *arXiv preprint arXiv:2311.08588*, 2023.

[48] N. Ashrafi, S. Bouktif, and M. Mediani, "Enhancing llm code generation: A systematic evaluation of multi-agent collaboration and runtime debugging for improved accuracy, reliability, and latency," *arXiv preprint arXiv:2505.02133*, 2025.

[49] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[50] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[51] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[52] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020.

[53] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh *et al.*, "Mixed precision training," *arXiv preprint arXiv:1710.03740*, 2017.

[54] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in neural information processing systems*, vol. 36, pp. 10 088–10 115, 2023.

[55] Microsoft Corporation, "Azure OpenAI Service," https://azure.microsoft.com/en-us/products/ai-services/openai-service, accessed: 2024-06-30.

[56] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[57] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[58] P. Bareiß, B. Souza, M. d'Amorim, and M. Pradel, "Code generation tools (almost) for free? a study of few-shot, pre-trained language models on code," *arXiv preprint arXiv:2206.01335*, 2022.

[59] D. Xu, T. Xie, B. Xia, H. Li, Y. Bai, Y. Sun, and W. Wang, "Does few-shot learning help llm performance in code synthesis?" *arXiv preprint arXiv:2412.02906*, 2024.

[60] T. Kadosh, N. Hasabnis, V. A. Vo, N. Schneider, N. Krien, M. Capota, A. Wasay, N. Ahmed, T. Willke, G. Tamir *et al.*, "Monocoder: Domain-specific code language model for hpc codes and tasks," *arXiv preprint arXiv:2312.13322*, 2023.

[61] A. Chaturvedi, D. Nichols, S. Singh, and A. Bhatele, "Hpc-coder-v2: Studying code llms across low-resource parallel languages," *arXiv preprint arXiv:2412.15178*, 2024.

[62] Z. Shengyu, D. Linfeng, L. Xiaoya, Z. Sen, S. Xiaofei, W. Shuhe, L. Jiwei, R. Hu, Z. Tianwei, F. Wu *et al.*, "Instruction tuning for large language models: A survey," *arXiv preprint arXiv:2308.10792*, 2023.

[63] T. Wu, R. Yang, J. Li, P. Hu, N. Wong, and Y. Yang, "Shadow-ft: Tuning instruct via base," *arXiv preprint arXiv:2505.12716*, 2025.

[64] torchtune maintainers and contributors, "torchtune: Pytorch's finetuning library," Apr. 2024. [Online]. Available: https//github.com/pytorch/torchtune

[65] L. Chen, A. Bhattacharjee, N. K. Ahmed, N. Hasabnis, G. Oren, B. Lei, and A. Jannesari, "Compcodevet: A compiler-guided validation and enhancement approach for code dataset," *arXiv preprint arXiv:2311.06505*, 2023.

[66] X. Wang, Y. Wang, Y. Wan, F. Mi, Y. Li, P. Zhou, J. Liu, H. Wu, X. Jiang, and Q. Liu, "Compilable neural code generation with compiler feedback," *arXiv preprint arXiv:2203.05132*, 2022.

[67] P. Jana, P. Jha, H. Ju, G. Kishore, A. Mahajan, and V. Ganesh, "Cotran: An llm-based code translator using reinforcement learning with feedback from compiler and symbolic execution," in *ECAI 2024*. IOS Press, 2024, pp. 4011–4018.

[68] L. Chen, B. Lei, D. Zhou, P.-H. Lin, C. Liao, C. Ding, and A. Jannesari, "Fortran2cpp: Automating fortran-to-c++ migration using llms via multi-turn dialogue and dual-agent integration," *arXiv preprint arXiv:2412.19770*, 2024.

[69] M. T. Dearing, Y. Tao, X. Wu, Z. Lan, and V. Taylor, "LASSI: An llm-based automated self-correcting pipeline for translating parallel scientific codes," *arXiv preprint arXiv:2407.01638*, 2024.

979-8-3315-7844-2/25/$31.00 ©2025 IEEE
Authorized licensed use limited to: Univ of East Anglia. Downloaded on April 25,2026 at 17:05:27 UTC from IEEE Xplore. Restrictions apply.